

Study 프로젝트 — 필수 프론트엔드 지식 정리

본 문서는 앞선 [frontend-flow.pdf](#) 의 화면 흐름을 이해하기 위해 실제로 알아야 하는 프론트엔드 핵심 개념만 모은 학습용 자료입니다.
대상: React 18 · React Router v6 · Vite · Fetch API + Session Cookie · Kakao Maps JS SDK.

목차

1. SPA 기본 — React · Vite · 진입점
2. JSX · 컴포넌트 · props · 단방향 데이터 흐름
3. React 상태 (state) 와 리렌더링 규칙
4. 핵심 Hooks — useState · useEffect · useRef · useMemo · useCallback
5. 제어 컴포넌트와 폼 처리
6. React Router v6 — Routes · useNavigate · useParams · useLocation
7. 라우트 가드 (RequireAuth) 패턴
8. 데이터 패칭 — Fetch API · credentials: 'include'
9. 세션 쿠키 · CORS · 자격 증명 동봉
10. 컴포넌트 간 통신 — 커스텀 이벤트([auth-changed](#)) · Context
11. StrictMode 더블 마운트 · useRef 가드
12. 상태 보존 — sessionStorage / localStorage / URL 쿼리
13. 외부 SDK 통합 — Kakao Maps
14. 환경변수 · Vite ([import.meta.env.VITE_*](#))
15. 빌드 / 배포 · dev 프록시
16. XSS / CSRF 등 프론트 보안 기초
17. UX 패턴 — 로딩 · 에러 · 빈 상태 · 토스트
18. 학습 체크리스트 / 다음 단계

1. SPA 기본 — React · Vite

1-1. SPA(Single Page Application)

최초 HTML 한 장만 받고, 이후 화면 전환은 JS 가 DOM 을 갈아치우는 방식. 서버 왕복이 사라져 빠르지만, 새로고침 시 라우터가 클라이언트 라우트를 못 찾는 문제가 있어 서버에 SPA fallback 설정([/index.html](#) 로 리라이트)이 필요합니다.

1-2. React 진입점

```
// main.jsx
import { createRoot } from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

- [createRoot](#) 는 React 18 의 동시성 모드를 활성화합니다.
- [StrictMode](#) 는 개발 환경에서만 동작하며, 의도적으로 컴포넌트를 두 번 마운트해 부수효과를 드러냅니다.

1-3. Vite

- 네이티브 ESM 기반 dev 서버 — webpack 보다 훨씬 빠른 cold start.
- 빌드는 Rollup 으로 번들링 → `dist/` 산출.
- `vite.config.js` 에서 dev 프록시·플러그인·alias 등을 설정.

2. JSX · 컴포넌트 · props

2-1. JSX 는 그냥 함수 호출

```
<Button color="primary">저장</Button>
// ↓ Babel/SWC 변환 결과
React.createElement(Button, { color: "primary" }, "저장")
```

즉, 컴포넌트는 **props** 라는 객체를 받아 **React 엘리먼트**를 반환하는 함수입니다.

2-2. props 와 단방향 데이터 흐름

- 부모 → 자식으로만 데이터를 내려보냅니다.
- 자식이 부모에게 알려려면 콜백(**props 함수**) 을 전달받아 호출합니다.
- 이 단방향성 때문에 데이터 흐름 추적이 쉬워집니다.

```
function NoticeList({ onSelect }) {
  return items.map(n =>
    <Row key={n.id} notice={n} onClick={() => onSelect(n.id)} />);
}
```

2-3. key 의 의미

리스트 렌더 시 **key** 는 React 가 항목의 동일성을 추적해 **DOM** 을 최소 변경하기 위한 식별자입니다.

안티패턴: `key={index}` . 순서가 바뀌면 잘못된 DOM 이 매핑되어 입력값/포커스가 영kip니다. 안정적인 ID를 쓰세요.

3. 상태(state) 와 리렌더링

3-1. 언제 리렌더링되는가

1. `setState` 가 호출됐을 때
2. 부모가 리렌더돼서 props 가 새로 내려올 때
3. `Context` 값이 바뀌어 구독 중인 컴포넌트일 때

3-2. 불변성

객체/배열 상태는 **새 객체**로 교체해야 합니다. 같은 참조를 그냥 변경하면 React 는 변화를 감지하지 못합니다.

```
// ❌ 안 됨
state.items.push(newItem);
setState(state);

// ✅ 새 배열로 교체
setItems(prev => [...prev, newItem]);
```

3-3. 상태 끌어올리기 (lifting state up)

두 형제 컴포넌트가 같은 데이터를 봐야 하면 **공통 부모**로 상태를 올립니다. 그 이상 떨어진 컴포넌트끼리는 Context 또는 이 프로젝트처럼 **커스텀 이벤트**를 사용합니다.

4. 핵심 Hooks

4-1. useState

```
const [user, setUser] = useState(null);
// 함수형 업데이트 - 직전 값에 의존할 때 안전
setUser(prev => ({ ...prev, nickname: '새 닉네임' }));
```

4-2. useEffect — 부수효과(데이터 패칭, 구독, 타이머 등)

```
useEffect(() => {
  let cancelled = false;
  fetch(`${API}/api/notices/${id}`, { credentials: 'include' })
    .then(r => r.json())
    .then(data => { if (!cancelled) setNotice(data); });
  return () => { cancelled = true; }; // 클린업: 언마운트/재실행 시
}, [id]); // 의존성 배열
```

- 의존성 배열을 생략하면 매 렌더마다, 빈 배열이면 마운트/언마운트 시 한 번, 값을 넣으면 그 값이 바뀔 때만 실행.
- 비동기 결과를 setState 하기 전에 언마운트/재실행 여부를 검사해야 메모리 누수 경고를 피합니다.

4-3. useRef

- DOM 직접 접근: `const inputRef = useRef(); <input ref={inputRef} /> .`
- 리렌더 없이 유지되는 값: 이 프로젝트의 `NoticeDetail` 에서 `viewedRef` 로 조회수 POST 1회 보장.

```
const viewedRef = useRef(false);
useEffect(() => {
  if (viewedRef.current) return;
  viewedRef.current = true;
  fetch(`${API}/api/notices/${id}/views`, { method: 'POST', credentials: 'include' });
}, [id]);
```

4-4. useMemo · useCallback

비용이 큰 계산이나 자식 컴포넌트의 props 안정성이 필요할 때만 사용. 기본은 쓰지 않는 것이 정답입니다 — 잘못 쓰면 오히려 느려집니다.

```
const filtered = useMemo(
  () => items.filter(it => it.name.includes(query)),
  [items, query]
);

const onSelect = useCallback((id) => navigate(`/notices/${id}`), [navigate]);
```

4-5. 커스텀 훅

이름이 `use` 로 시작하고 내부에서 다른 훅을 사용하는 함수. 반복되는 로직(예: `useAuth` , `useDebounce`)을 캡슐화.

5. 제어 컴포넌트와 폼 처리

```
function LoginForm({ onLogin }) {
  const [form, setForm] = useState({ username: '', password: '' });
```

```

const change = (e) => setForm({ ...form, [e.target.name]: e.target.value });

const submit = async (e) => {
  e.preventDefault(); // 기본 폼 제출 차단
  try { await onLogin(form); }
  catch (err) { setError(err.message); }
};

return (
  <form onSubmit={submit}>
    <input name="username" value={form.username} onChange={change} required />
    <input name="password" type="password" value={form.password} onChange={change} required />
    <button type="submit">로그인</button>
  </form>
);
}

```

- **제어 컴포넌트**: value 와 onChange 로 React state 가 폼 값을 완전히 통제 — 검증/조건부 제출/리셋이 쉬움.
- `e.preventDefault()` 를 안 부르면 페이지 전체가 새로 고침되며 SPA 상태가 사라집니다.

6. React Router v6

6-1. 라우트 정의

```

<Routes>
  <Route path="/" element={<RequireAuth><NoticeList /></RequireAuth>} />
  <Route path="/notices/:id" element={<RequireAuth><NoticeDetail /></RequireAuth>} />
  <Route path="/forgot" element={<PasswordForgot />} />
  <Route path="*" element={<NotFound />} /> // 404 fallback
</Routes>

```

6-2. 네비게이션 혹

혹 / 컴포넌트	용도
<code>useNavigate()</code>	코드에서 페이지 이동. <code>navigate('/notices/1')</code> , <code>navigate(-1)</code> .
<code>useParams()</code>	URL 경로 변수 읽기. <code>const { id } = useParams()</code> .
<code>useSearchParams()</code>	쿼리스트링 읽기/수정. <code>?token=...</code> .
<code>useLocation()</code>	현재 pathname/search/state.
<code><Link to="..."></code>	a 태그 대신, 전체 새로고침 없이 라우트 전환.
<code><Navigate to="/login" replace /></code>	선언적 리다이렉트.

6-3. Link 와 a 의 차이

`<a href>` 는 브라우저가 새 페이지를 받아오면서 **SPA 상태와 메모리가 초기화**됩니다. 내부 라우트는 반드시 `<Link>` 또는 `navigate()` 를 쓰세요.

7. 라우트 가드 (RequireAuth) 패턴

이 프로젝트의 `RequireAuth` 는 단순한 컴포넌트로 만든 게이트입니다.

```
function RequireAuth({ children }) {
  const [state, setState] = useState('loading'); // 'loading' | 'authenticated' | 'guest'

  const check = useCallback(() => {
    fetch(`${API}/api/auth/me`, { credentials: 'include' })
      .then(r => setState(r.ok ? 'authenticated' : 'guest'))
      .catch(() => setState('guest'));
  }, []);

  useEffect(() => {
    check();
    const onAuthChanged = () => check();
    window.addEventListener('auth-changed', onAuthChanged);
    return () => window.removeEventListener('auth-changed', onAuthChanged);
  }, [check]);

  if (state === 'loading') return <Spinner />;
  if (state === 'guest') return <LoginRequiredCard />;
  return children;
}
```

포인트:

- 3-state 머신 (loading / authenticated / guest) 으로 깜빡임 없이 표시.
- 전역 이벤트 `auth-changed` 를 구독해 로그인/로그아웃 직후 자동 재검증.
- 리스너 등록 시 반드시 클린업으로 해제 (메모리 누수 방지).

8. 데이터 패칭 — Fetch API

8-1. 기본 패턴

```
const res = await fetch(`${API}/api/notices`, {
  method: 'POST',
  credentials: 'include', // 세션 쿠키 동봉
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ title, content }),
});

if (!res.ok) {
  const text = await res.text();
  let msg = text;
  try { msg = JSON.parse(text).message || text; } catch {}
  throw new Error(msg || `HTTP ${res.status}`);
}

return await res.json();
```

8-2. 자주 빠지는 함정

- `fetch` 는 4xx/5xx 를 `reject` 하지 않습니다. 직접 `res.ok` 를 검사해야 합니다.
- `Content-Type` 을 빠뜨리면 백엔드가 본문을 파싱하지 못해 400 이 납니다.

- 큰 응답을 `text()` + `json()` 둘 다 부르면 `stream` 이 이미 소비됐다는 에러가 납니다 — 둘 중 하나만.
- 요청 취소가 필요하면 `AbortController` + `signal` .

9. 세션 쿠키 · CORS

9-1. credentials 옵션

값	의미
<code>omit</code> (기본)	쿠키 보내지 않음.
<code>same-origin</code>	같은 출처일 때만 보냄.
<code>include</code>	다른 출처여도 보냄. 이 프로젝트가 사용. 서버의 <code>Access-Control-Allow-Credentials: true</code> 와 짝.

9-2. CORS 와 자격 증명

- 프론트(`localhost:5173`)와 백엔드(`localhost:8080`)는 다른 출처입니다.
- 그래서 백엔드는 CORS 응답 헤더로 출처와 자격 증명 허용을 명시해야 합니다(`backend-knowledge` §10 참고).
- `credentials=include` 인 요청에서는 `Access-Control-Allow-Origin: *` 가 거부됨 — 백엔드가 정확한 출처를 응답해야 함.

9-3. preflight (OPTIONS) 요청

커스텀 헤더(`Content-Type: application/json` 포함)나 GET/POST 외 메서드는 브라우저가 먼저 `OPTIONS` 로 허용 여부를 확인합니다. 백엔드 로그에 `OPTIONS` 가 보이는 이유.

10. 컴포넌트 간 통신 — 커스텀 이벤트 · Context

10-1. 이 프로젝트의 선택: window 커스텀 이벤트

```
// 로그인 성공 후
setUser(data);
window.dispatchEvent(new Event('auth-changed'));

// 다른 컴포넌트 (RequireAuth)
useEffect(() => {
  const onChange = () => recheck();
  window.addEventListener('auth-changed', onChange);
  return () => window.removeEventListener('auth-changed', onChange);
}, []);
```

장점: 트리에서 멀리 떨어진 컴포넌트도 의존성 없이 통신.

단점: 타입 안전성 부족, 디버깅 어려움.

10-2. 일반적인 대안 — Context API

```
const AuthContext = createContext(null);

function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const value = useMemo(() => ({ user, setUser }), [user]);
  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

function useAuth() { return useContext(AuthContext); }
```

규모가 커지면 Context + 커스텀 혹은 정리하는 게 유지보수에 유리합니다. 더 커지면 React Query/SWR(서버 상태) + Zustand/Redux(클라이언트 상태) 분리.

11. StrictMode 더블 마운트와 useRef 가드

React 18 + StrictMode 는 개발 환경에서 컴포넌트를 **마운트** → **언마운트** → **다시 마운트** 시킵니다. 그래서 마운트 시 `POST /views` 같은 부수효과는 두 번 호출될 수 있습니다.

가드 패턴 1 — useRef

```
const did = useRef(false);
useEffect(() => {
  if (did.current) return;
  did.current = true;
  doOnce();
}, []);
```

가드 패턴 2 — 서버에서 멍등성 보장

조회수 같은 증가 연산은 클라이언트로 막기보다 서버에서 IP/세션/시간 윈도우 기준으로 처리하는 것이 정석.

12. 상태 보존

저장소	수명	용도 예
<code>useState</code>	컴포넌트 마운트 동안	입력값, UI 토글
<code>sessionStorage</code>	탭이 살아있는 동안	NoticeList 검색조건 복원 (이 프로젝트)
<code>localStorage</code>	도메인이 같으면 영구	다크모드, 최근 검색어
URL 쿼리	URL 이 살아있는 동안	공유 가능한 검색/페이지 상태
쿠키 (HttpOnly)	서버 설정	세션 ID (JS 에서 접근 불가)

가능하면 **URL 쿼리**를 1순위로 쓰세요. 공유·뒤로가기·새로고침에 모두 잘 동작합니다. 새로고침까지 유지가 필요 없으면 메모리 (state) 만으로 충분.

13. 외부 SDK 통합 — Kakao Maps

외부 JS SDK 는 **전역 객체(window.kakao)** 로 노출되는 경우가 많습니다. React 와 어울리려면 다음을 지킵니다.

```
useEffect(() => {
  if (!window.kakao || !window.kakao.maps) return;
  window.kakao.maps.load(() => {
    const map = new window.kakao.maps.Map(containerRef.current, {
      center: new window.kakao.maps.LatLng(37.4892775, 126.7246513),
      level: 4,
    });
    mapRef.current = map; // ref 로 React 상태 밖에 보관
  });
  return () => { /* 필요 시 리스너 해제 */ };
}, []);
```

- SDK 가 만든 마커/InfoWindow 는 **React 가상 DOM 밖에 존재** → 직접 `setMap(null)` 으로 정리해야 누수 없음.
- 로딩 타이밍 — `index.html` 의 `<script>` 태그 또는 동적 삽입. 준비 전 호출 방지를 위해 `kakao.maps.load()` 로 감쌉니다.

- API 키는 도메인 제한이 있는 JS 키만 프론트에 둡니다. REST API 키는 절대 프론트 노출 금지(이 프로젝트는 백엔드 프록시로 처리).

14. 환경변수 — Vite

```
# frontend/.env.local
VITE_API_BASE=http://localhost:8080
VITE_KAKAO_JS_KEY=abcdef...
```

```
// 코드에서
const API = import.meta.env.VITE_API_BASE;
```

- 접두사 `VITE_` 가 붙은 것만 클라이언트 번들에 포함됩니다 (실수로 비밀 키가 노출되는 것을 방지).
- 그래도 번들된 값은 클라이언트가 볼 수 있다는 사실은 잊지 마세요. 진짜 비밀 키는 백엔드로 옮기는 것이 정답.

15. 빌드 / 배포 · dev 프록시

15-1. 명령어

명령	역할
<code>npm run dev</code>	Vite dev 서버. HMR(핫 리로드) 지원.
<code>npm run build</code>	<code>dist/</code> 에 정적 산출물 생성.
<code>npm run preview</code>	빌드 결과를 로컬에서 정적 서빙해 확인.

15-2. dev 프록시 (선택)

CORS 설정을 매번 손대기 싫다면 Vite 가 `/api` 요청을 백엔드로 프록시하게 만들 수 있습니다.

```
// vite.config.js
export default defineConfig({
  server: {
    proxy: {
      '/api': { target: 'http://localhost:8080', changeOrigin: true }
    }
  }
});
```

15-3. 배포 시 SPA fallback

Nginx 예:

```
location / {
  try_files $uri /index.html;
}
```

이게 없으면 사용자가 `/notices/123` 에서 새로고침할 때 404 가 납니다.

16. 프런트 보안 기초

16-1. XSS — 가장 흔한 위협

- React 는 기본적으로 텍스트 출력을 escape 하므로 **JSX 만으로는 거의 안전합니다**.
- 위험한 지점은 `dangerouslySetInnerHTML` . 사용자 입력을 그대로 넣으면 안 되고, 서버에서 sanitize 한 안전한 HTML 만 넣어야 합니다.
- URL 출력 시 `javascript:` 스킴 차단 — `href={userUrl}` 도 검증 후 렌더.

16-2. CSRF — 세션 쿠키 기반에서 신경 써야 함

- 공격자가 만든 페이지에서 우리 사이트로 요청을 보내면 브라우저가 **쿠키를 자동으로 동봉**합니다.
- 방어: **SameSite=Lax/Strict** 쿠키, CSRF 토큰, **출처 검증**(Origin/Referer 헤더), JSON Content-Type 요구.
- 이 프로젝트는 SameSite=Lax + CORS 출처 화이트리스트 + JSON 요구로 실용적인 방어선을 만들고 있음.

16-3. 비밀번호 등 민감 정보

- 입력 후 메모리에 오래 두지 말 것. 제출 후 state 초기화.
- 비밀번호를 URL/쿼리/local storage 에 절대 저장 금지.
- `autocomplete="new-password"` , `type="password"` 사용.

17. UX 패턴 — 로딩 · 에러 · 빈 상태 · 토스트

화면 컴포넌트는 거의 항상 **4가지 상태**를 가집니다.

1. **idle / loading** — 스피너 또는 스켈레톤.
2. **success** — 데이터 렌더.
3. **empty** — "결과가 없습니다" 안내 (배경/일러스트로 차별하게).
4. **error** — 빨간 배너 또는 토스트 + 재시도 버튼.

이 네 가지 모두를 처음부터 설계하면 사용자가 한 번도 막막하지 않습니다.

```
if (loading) return <Spinner />;
if (error) return <ErrorBanner message={error} onRetry={refetch} />;
if (!items.length) return <EmptyState title="결과가 없습니다" />;
return <List items={items} />;
```

17-1. 무한 로딩 방지

- `fetch` 후 `finally` 에서 `setLoading(false)` .
- 요청 자체에 타임아웃을 거는 것이 안전 — `AbortController` + `setTimeout` .

17-2. 더블 클릭 / 중복 제출 방지

제출 버튼은 처리 중 `disabled` 로 잠그고, 결제처럼 위험한 액션은 먹등기를 같이 보내는 패턴을 검토합니다.

18. 학습 체크리스트

코드를 따라 읽으며 답해 보세요

- `RequireAuth` 가 `auth-changed` 이벤트에 의존하는 이유는? Context 로 대체하면 무엇이 달라질까?
- `NoticeDetail` 의 `viewedRef` 가 없으면 어떤 문제가 생기는가?
- `fetch` 에서 `credentials: 'include'` 를 빼면 어떤 동작이 깨지는가?

- NoticeList 의 검색조건을 `sessionStorage` 가 아니라 URL 쿼리에 두면 무엇이 좋아지나?
- Kakao Maps 마커를 React state 에 보관하지 않고 `ref` 에 두는 이유는?

면접·코드 리뷰에서 자주 나오는 질문

- React 의 단방향 데이터 흐름이란? 왜 중요한가?
- `useEffect` 의 클린업 함수가 필요한 시나리오 3가지?
- `useMemo` / `useCallback` 을 남용하면 왜 더 느려질 수 있는가?
- SPA 에서 새로고침 시 404 가 나는 원리와 해결책?
- XSS 와 CSRF 의 차이와 각각의 방어 기법?
- fetch 가 4xx 에서 reject 하지 않는 이유와 그로 인한 실수?
- 제어 컴포넌트 vs 비제어 컴포넌트의 트레이드오프?

다음 단계

- **TanStack Query (React Query) / SWR** — 서버 상태 캐싱·재검증·낙관적 업데이트. fetch 보일러플레이트가 크게 줄어듭니다.
- **TypeScript** — DTO 타입 안전성, 자동완성, 리팩토링 안정성.
- **Suspense + Error Boundary** — 로딩/에러를 컴포넌트 트리 단위로 선언적으로 처리.
- **React Hook Form / Zod** — 폼 검증을 선언적·타입 안전하게.
- **Testing Library + Vitest** — 사용자 행동 기반의 컴포넌트 테스트.
- **접근성(a11y)** — semantic HTML, ARIA, 키보드 내비게이션.
- **성능** — 코드 스플리팅(`React.lazy`), 이미지 lazy load, Lighthouse 검사.

끝. 본 문서는 `frontend-flow.pdf` 의 화면 흐름을 이해하기 위한 학습 자료입니다. 실제 구현과 차이가 있으면 코드를 우선으로 하고 본 문서를 갱신하세요.